Building Backend Systems

A Practical Curriculum

46 days. One concept at a time. Built for anyone who wants to understand backend engineering at the level of systems — not just frameworks.

Primary stack

Python · FastAPI · PostgreSQL · Redis · Docker

Capstone

KasiCredit — an AI-assisted lending and credit scoring API for the African informal economy

Preface

There is a difference between knowing how to use a framework and understanding the system underneath it. Most backend developers learn the framework. This curriculum teaches the system.

You will spend the first day writing a server using nothing but Python's raw socket library. No FastAPI. No Django. No framework of any kind. That is not a gimmick — it is the point. When you understand what a server is at the level of the operating system, every framework you encounter for the rest of your career becomes a familiar concept in unfamiliar syntax. When you do not understand it, every framework feels like magic, and magic is fragile.

This curriculum moves in the same order that requests move through a real production system. You start at the network layer and work outward — through HTTP, API design, databases, indexing, concurrency, authentication, caching, deployment, and finally AI integration. Each concept is introduced only when you have enough context for it to make sense. Design patterns, for example, do not appear until Week Four, when your codebase is complex enough that you will recognise the problems they solve without being told what those problems are.

The capstone is a real product — KasiCredit, a lending and credit scoring API built for the African informal economy. It uses mobile money transaction history to build credit profiles for people who have no formal credit record. It is not a toy project. It is designed to be the kind of thing you would put in a portfolio and be asked questions about in an interview.

If you are reading this as someone else's curriculum — someone shared this with you, or you found it somewhere — you are welcome here. Nothing about this document assumes prior knowledge beyond basic Python. It does

assume you are willing to build things before you fully understand them and to sit with discomfort until understanding arrives. That is the only prerequisite.

WHO THIS IS FOR

This curriculum was written for anyone who has written Python code before, has built something small — a script, a simple API, a project for a class — and wants to understand backend engineering at a deeper level than tutorials typically go.

It is specifically useful if you recognise yourself in any of the following: you have used FastAPI or Django but could not explain what happens between a request arriving and a response being sent; you have written database queries but could not explain why some queries are fast and others are slow; you have copied authentication code from a tutorial without fully understanding what a JWT is or why it expires; you have deployed something to a cloud service without understanding what Docker is actually doing.

This curriculum will not make you a senior engineer in 46 days. What it will do is give you the mental models that senior engineers use — models for thinking about systems, tradeoffs, failure modes, and design decisions. Those models are what separate engineers who debug by understanding from engineers who debug by guessing.

THE PHILOSOPHY BEHIND THIS CURRICULUM

Systems over frameworks

Frameworks come and go. The concepts underneath them do not. HTTP has not changed in its fundamentals since 1991. Relational databases work the same way they did in the 1970s. Concurrency problems are the same problems whether you are writing Python, Go, Java, or Rust. This curriculum spends most of its time on concepts that will still be relevant in twenty years and only uses FastAPI as the vehicle because it is the clearest Python framework for explaining what is happening underneath.

Depth before breadth

It is more valuable to understand one thing completely than to have a passing familiarity with ten things. When a topic appears in this curriculum, it appears with enough depth that you could reason about it under pressure — in an interview, in a production incident, in a code review. Breadth comes naturally once depth is established. The reverse is not true.

Building as the primary mode of learning

Every day has a practical section that is longer than the theory section. This is deliberate. You do not understand networking by reading about networking. You understand it by writing a server that breaks in a specific way and then fixing it. The experiments in the practical sections are as important as the explanations — in many cases more important.

The test that runs every day

After every session, ask yourself one question: if the framework I used today disappeared tomorrow, would I still understand what I built? If the answer is yes, you learned backend engineering. If the answer is no, go deeper on the concept before moving to the next day. There is no penalty for taking two days on a single topic. There is a cost to moving forward with a shaky foundation.

HOW EACH DAY WORKS

Every day is ninety minutes. The structure is always the same.

> **Theory · 20 minutes**
> Read and understand the concept before writing any code. This is not optional. Code written without conceptual grounding is code that cannot be debugged or extended. The theory sections are written to be read once slowly, not skimmed.

> **Practical · 60 minutes**

> Build the exercise. Type code by hand rather than copying and pasting — the act of typing forces you to read every character, and reading every character is where understanding comes from. The experiments in the practical sections are designed to break things deliberately. Breaking things deliberately is how you learn what holds systems together.

> **Reflection · 10 minutes**
> Answer the questions at the end of the day. You do not need to write anything down unless you want to. The goal is to find the edges of your understanding — the places where an explanation feels thin or a question feels unanswerable. Those edges are where tomorrow's learning begins.

Every Sunday is a summary day. No new concept is introduced. You write a short paragraph — five to ten sentences — about what clicked during the week and what did not. Then you push all your code to GitHub. The summary is not assessed by anyone. It is for you.

ENVIRONMENT SETUP

Before Day 1, you need the following installed and working on your machine. This section tells you what to install and why each tool is needed. Do not skip any of it — later days assume everything here is already in place.

Python 3.11 or later

The primary language of this curriculum. If you are on Windows, install Python from python.org and make sure to check the box that adds Python to your PATH during installation. Verify it is working by opening a terminal and running:

```
python --version
```

PostgreSQL 15 or later

The relational database used throughout the curriculum. On Windows, download the installer from postgresql.org. During installation, set a password for the postgres superuser and keep note of it — you will use it on Day 8. Verify the installation by opening a terminal and running psql --version.

Redis

An in-memory data store used for caching and background job queuing in Weeks Four and Five. On Windows, Redis does not have an official native installer. Install it through WSL — Windows Subsystem for Linux. Open PowerShell as administrator and run:

```
wsl --install
# then inside WSL:
sudo apt install redis-server
```

Docker Desktop

Used from Week Five onward to containerise your application and run multi-service stacks. Install Docker Desktop from docker.com. On Windows, Docker Desktop requires WSL 2 as its backend — the Redis installation above will have already set this up. Verify by running docker --version in a terminal.

Git and a GitHub account

Every day ends with a commit and a push to GitHub. Create an account at github.com if you do not have one. Install Git from git-scm.com. Configure your name and email:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Postman

A tool for sending HTTP requests to your API without needing a frontend. You will use it from Day 1. Download the desktop application from postman.com. The free tier is sufficient for everything in this curriculum.

A code editor

Visual Studio Code is recommended. Install the Python extension and the REST Client extension. If you have a strong preference for another editor, use it — nothing in this curriculum is editor-specific.

Your project repository

Create a folder called building-backend-systems on your machine. Inside it, create a folder for each week — week-01, week-02, and so on. Initialise a single Git repository at the root. Every file you write during this curriculum lives here. Create the repository on GitHub and push the empty folder structure before Day 1 begins.

PYTHON PACKAGES YOU WILL USE

You do not need to install all of these now. They are listed here so you know what is coming and can install each package on the day it is needed. The day number in parentheses indicates when each package first appears.

Package	First used	Purpose
`fastapi, uvicorn`	Day 2	The web framework and its ASGI server
`pydantic`	Day 5	Data validation and serialisation
`psycopg2-binary`	Day 8	PostgreSQL driver — connects Python to your database
`sqlalchemy`	Day 18	The ORM used to query your database through Python objects
`python-dotenv`	Day 19	Loads environment variables from a .env file
`python-jose`	Day 20	JWT creation and verification
`passlib[bcrypt]`	Day 20	Secure password hashing
`redis`	Day 23	Python client for Redis
`celery`	Day 28	Background task queue

`pytest, httpx`	Day 33	Testing framework and async HTTP client for test requests
`faker`	Day 15	Generates realistic fake data for seeding large tables
`chromadb`	Day 37	Local vector database for storing and querying embeddings
`anthropic`	Day 38	Official Python SDK for the Anthropic API
`prometheus-fastapi-instrumentator`	Day 40	Adds Prometheus metrics to FastAPI automatically

THE CAPSTONE: KASICREDIT

The curriculum ends with a three-day capstone project — a product called KasiCredit. It is a lending and credit scoring API built for the African informal economy, specifically for small business owners and individuals who have no formal credit history.

In Congo-Brazzaville, Ghana, and across much of sub-Saharan Africa, the majority of economically active people are excluded from formal lending because they have never held a bank account, never had a credit card, and never appeared in a credit bureau's records. They are not poor — many run profitable small businesses — but they are invisible to traditional credit systems. KasiCredit builds a credit profile from the data they do have: mobile money transaction history.

The product uses two AI features, both deliberately narrow in scope. The first generates a plain-language explanation of each credit decision — something a loan officer without a technical background can read and act on. The second flags applications where the stated income is inconsistent with the transaction history, routing them to human review rather than automated rejection. Every other part of the system is deterministic business logic, not AI. Credit decisions that affect people's livelihoods must be auditable.

The capstone is designed to use every major concept from the preceding 43 days: layered architecture, PostgreSQL with proper indexing, JWT authentication with role-based access control, Redis caching, background jobs, structured logging, Docker Compose with multiple services, a CI

pipeline, and an AI integration layer. It is not a toy project. It is the kind of thing you would put in a portfolio and be asked detailed questions about.

THE 46-DAY MAP

The table below is a complete map of the curriculum. Each week has a unifying theme. Day 7, 14, 21, 29, 36, and 43 are summary days — no new concept, only consolidation and a GitHub push.

Week One — How the Internet Talks to Your Code	
Day 1	What Is a Server, Really?
Day 2	HTTP: The Language of the Web
Day 3	HTTP Methods and Status Codes
Day 4	REST API Design
Day 5	JSON and Pydantic
Day 6	Application Architecture: Layers
Day 7	Week One Summary
Week Two — Databases: Where Data Actually Lives	
Day 8	Relational Databases from First Principles
Day 9	SQL: Reading Data
Day 10	SQL: Combining Data with JOINs
Day 11	SQL: Aggregation and Grouping
Day 12	SQL: Subqueries and Advanced Patterns
Day 13	Schema Design and Normalization
Day 14	Week Two Summary
Week Three — Making Databases Fast and Safe	
Day 15	Indexes and How Databases Find Data
Day 16	Transactions and ACID
Day 17	Concurrency: Race Conditions and Locks
Day 18	ORMs: SQLAlchemy
Day 19	Environment Variables and Secrets
Day 20	Authentication: Sessions and JWT
Day 21	Week Three Summary
Week Four — Production-Grade Backend Engineering	

Day 22	Authorization: Roles and Permissions
Day 23	Caching with Redis
Day 24	Error Handling
Day 25	Design Patterns for Backend Systems
Day 26	Logging
Day 27	API Security
Day 28	Background Jobs
Day 29	Week Four Summary
Week Five — Infrastructure and Deployment	
Day 30	Async Programming
Day 31	Docker
Day 32	Nginx as a Reverse Proxy
Day 33	Testing
Day 34	CI/CD with GitHub Actions
Day 35	WebSockets
Day 36	Week Five Summary
Week Six — AI Integration and Modern Data Systems	
Day 37	Vector Databases and Embeddings
Day 38	RAG Architecture
Day 39	Tool Calling and Agents
Day 40	Observability: Metrics and Monitoring
Day 41	Message Queues: RabbitMQ
Day 42	System Design Thinking
Day 43	Week Six Summary
Capstone — KasiCredit: A Fintech Lending API	
Day 44	Capstone: Design Day
Day 45	Capstone: Build Day One
Day 46	Capstone: Build Day Two and Final Summary

A NOTE ON PACE

Ninety minutes a day is the recommended session length. That said, some days will take longer — and that is not failure. The database week in particular (Days 8 through 14) contains concepts that compound on each other. If you

find yourself spending two days on indexing because the query plan output is not making sense yet, spend two days on it. The curriculum will not expire.

What matters more than pace is that you do not move forward with a shaky foundation. A useful heuristic: before moving to the next day, you should be able to explain the current day's core concept out loud, in plain English, to someone who has never programmed. If you cannot do that, the understanding is not yet solid enough to build on.

The weekly summary days are also rest days in the sense that no new technical content is introduced. Use them to catch up on anything from the week that felt rushed, to clean up and document your code, and to write the summary paragraph. Do not skip them.

ONE LAST THING BEFORE YOU BEGIN

Backend engineering is not a collection of tools. It is a way of thinking about systems — about how data moves, where it lives, what happens when two things try to change it at the same time, how a system behaves when part of it fails, and how you know when something is wrong before your users tell you.

The frameworks you will learn are expressions of that thinking, not substitutes for it. FastAPI is a well-designed expression. Spring Boot is a different expression of many of the same ideas. Redis, PostgreSQL, Docker — each is a tool that encodes decades of systems thinking into something you can install and run. Your job as an engineer is to understand what that thinking is, not just to know which commands to type.

That understanding takes time. It is built gradually, through building things that break, reading error messages carefully, and asking why at every step. This curriculum gives you a structure for that process. The understanding itself is yours to build.

Day 1 begins on the next page.

> **Ready to begin?**
> Complete the environment setup in the previous section if you have not done so.
>
> Create your GitHub repository and push the empty folder structure.
>
> Open your terminal. Turn the page.